

Implementation of FFT Algorithm for FPGA on MATLAB

Aim

This report aims to observe how one could implement an Fast Fourier Transform (FFT) algorithm for FPGAs on MATLAB and Simulink. This report will also focus on the block-based verification of FFT algorithm regarding the digital signal processing (DSP) principals. To achieve such application, in this report, one can find automated Hardware Description Language (HDL) generation useful. Then, this report aims to inspect the resource management on the chip that is emerged from the synthesized code.

HDL Code Generation and FPGA Synthesis from Simulink Model

The integrated functions of MATLAB are software based, meaning that the computation is done sequentially and MATLAB uses highly sensitive floating point numbers. On the other hand, FPGAs process the data in a parallel manner at each clock cycle. The inherited design of an FPGA is hardware-targeted that saves space and time. Hence, it uses fixed point format. DSP principles distribute the complex mathematical labor into small toolbox blocks enabling the FPGA work without a problem at high frequencies.

Normally, coding a counter algorithm directly to the FPGA does not require labor and time. However, It might invite the probability of making detrimental errors throughout the process in large scaled projects. This problem is solved by the Intellectual Property (IP) of the toolbox.

The given MATLAB example [1], shows the basics of how one should be enabled by the preliminary information on HDL code generation in Simulink. Here the Simulink HDL Workflow Advisor is introduced that comes handy in diverse manners.

The example is the implementation of a 4-bit counter that counts upwards when enable is '1' and saves the last number otherwise. Having used the command `openExample("hdlcoder/SimpleUpCounterModelHDLCodeExample")`, one can see the schematic for the 4-bit counter.

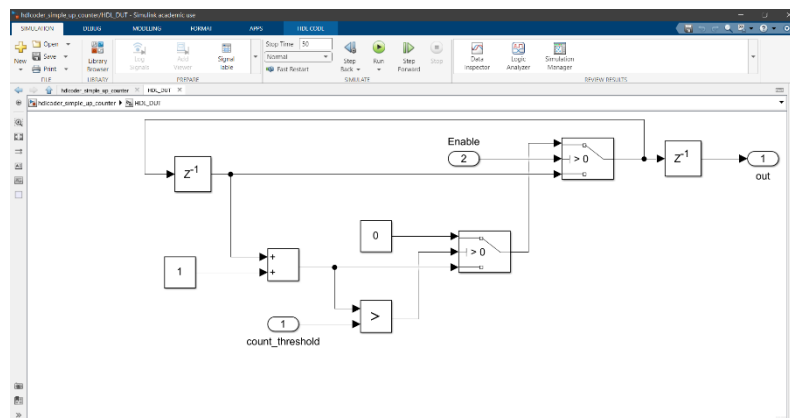


Figure.1: Simulink schematic of the 4-bit counter

To synthesize the simple circuit in *Figure.1*, one should introduce the FPGA Design IDE to MATLAB before using built-in HDL Workflow Advisor. I use the version 2022.2 of Xilinx Vivado so I simply write the command “hdlsetuptoolpath('ToolName','Xilinx Vivado','ToolPath',...

'D:\\Vivado\\2022.2\\bin\\vivado.bat');” to let MATLAB and Simulink be aware of the FPGA design IDE.

Then, one should open built-in HDL Workflow Advisor by first clicking “HDL coder” and then “HDL Workflow Advisor “ as in *Figure.2*.

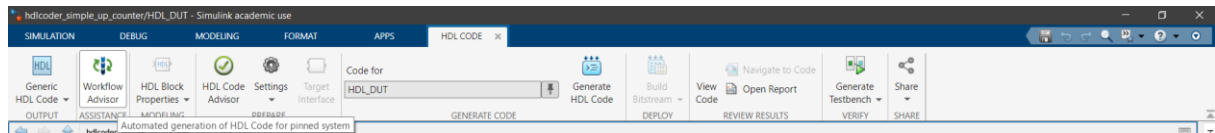


Figure.2: Path of HDL Workflow Advisor

Generating HDL code:

First step is to set Target Device and Synthesis Tool. Since I already introduced Xilinx Vivado to MATLAB, I saw the program name there, and there the program asks about the specifications of the FPGA that will be used. This is a mandatory step since the program will show how much space is used and optimize accordingly.

Then the clock frequency is asked and as in the example, I have clocked it at 200MHz. also Vivado will use this time constraint to have no delays (positive slack time) while routing the FPGA.

After that the validity of the blocks and sampling time is checked and the language of desire is asked.

Having verified the above steps, Register Transfer Level (RTL) code is written for that simple model.

Performing FPGA Synthesis and Analysis

Having verified the FPGA and its IDE, I ran the implementation and as it can be seen from *Figure.3* it was successful. Since the example is a simple basic counter circuit, it only took 12 Configurable Logic Block Look-Up Table (CLB LUT) and 8 CLB Registers.

Furthermore, for a target clocking speed 200Mhz (5ns), the maximum delay is 1.723ns. This resulted in a positive slack time of 3.257ns meaning that all the signals arrived 3.257ns earlier. This also means that with clock frequency up to 573.72MHz, I wouldn't get any issue implementing.

Resource summary			
Resource	Usage	Available	Utilization (%)
CLB LUTs	12	44000	0.03
CLB Registers	8	88000	0.01
DSPs	0	400	0.00
Block RAM Tile	0	100	0.00
URAM	0	0	

Parsed timing report file: [timing_post_route.rpt](#).

Timing summary	
	Value
Requirement	5 ns (200 MHz)
Data Path Delay	1.723 ns
Slack	3.257 ns
Clock Frequency	573.72 MHz

Figure.3: Resource and Time summary tables after the implementation

Looking at *Figure.4* one could see that the delay is so low that the critical path only occurs when system is activated. And the delay value of 0.05ns tells that there wouldn't be a bottleneck for conducting counting.

```
=====
Delay_out1_reg[2] | gm_hdlcoder_simple_up_counter/HDL_DUT/Delay | 0.0580 | true
=====
```

Figure.4: Delay of the system

Implementing FFT Algorithm for FPGA

Having done the validation of HDL code generation successfully, this study alters its focus to integrating FFT algorithm on FPGA. Another given example is showing one how this could be done. [2] DSP system toolbox uses floating point that computer understands well but couldn't be embedded to chips directly. For hardware as FPGAs, one must use streaming data interfaces with fixed point data types. DSP HDL Toolbox handles this and arrives gigasamples-per-second (GSPS).

There is two main IP architectures to achieve the whole process. If the data is in vast amounts and one needs haste in processing the data, they should use Streaming Radix 2² architecture that is capable of GSPS with a huge amount of storage usage. On the other hand, if the capacity to the FPGA is limited, one should use Burst Radix 2 architecture that uses less storage and more time.

Streaming Radix 2² Architecture

Contemporary ADCs work at GSPS rates. However, FPGAs are exposed to a bottleneck even at MHz's. this emerges an insufficiency. To overcome this one should use vectorized inputs to make computations much faster in a parallel manner, reducing clock rates. Here radix 2² turns the input into an 8x1 vector.

```
FFTLength = 512;
```

The input data is two sine waves, 200 KHz and 250 KHz, each sampled at 1*2e6 Hz. The input vector size is 8 samples.

```
FrameSize = 8;
```

```
Fs = 1*2e6;
```

To demonstrate the use of a **valid** control signal for noncontinuous input data, this example applies valid input every other cycle.

```
ValidPattern = [1,0];
```

Figure.5: FFT variables

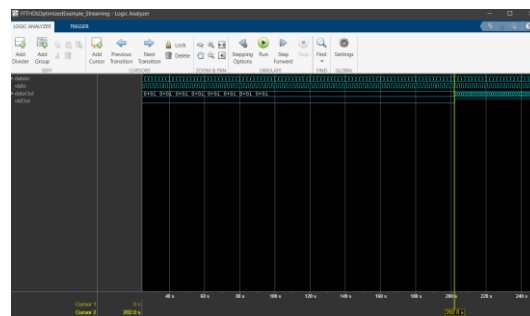


Figure.6: Benchmark result of Radix 2² Architecture

Burst Radix 2 architecture

Burst rather uses significantly less space, and as a result of it, 512 FFT inputs have to be processed in that shrink architecture taking more time

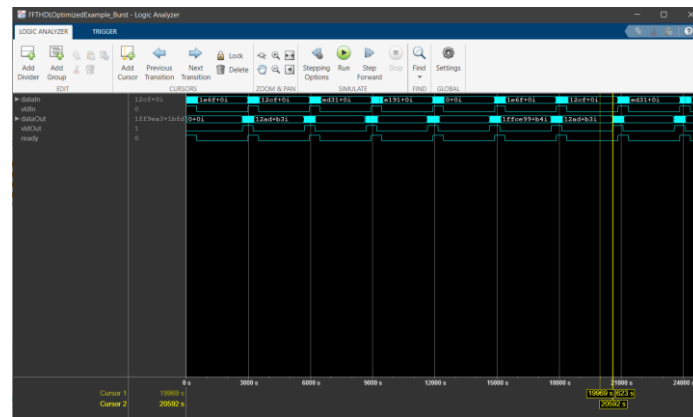


Figure.7: Result of Radix 2 Architecture

Conclusion

This report successfully demonstrated Implementation of FFT Algorithm for FPGA on MATLAB. Firstly, automated HDL workflow was verified to generate code and synthesize logic without bottlenecks for the 4-bit counter implementation part. From the two different hardware implementation architectures using the DSP HDL Toolbox, an inevitable trade-off of size and speed has been addressed. As it can be seen from *Figure.6* and *Figure.7*, the streaming architecture exhibited a drastically shorter delay than burst, explaining the space-time trade-off. In conclusion there is not a universal solution to testing the system. Engineering takes a major place in determining which approach is optimal to proceed with

REFERENCES

- [1] "HDL Code Generation and FPGA Synthesis from Simulink Model." [Online]. Available: <https://www.mathworks.com/help/hdlcoder/gs/fpga-synthesis-and-analysis-using-the-hdl-workflow-advisor.html>. Accessed: August 7, 2026.
- [2] "Implement FFT Algorithm for FPGA." [Online]. Available <https://www.mathworks.com/help/dsphdl/gs/generate-hdl-code-for-fft-hdl-optimized-block.html>. Accessed: August 7, 2026.